



IMAP SGF — Gestão Financeira

O núcleo **financeiro** do ERP interno: vendas, pagamentos, boletos, faturas, depósitos, remessas bancárias e comissões. **De ColdFusion + Windows Server → Python/FastAPI + Astro/Vite + Docker. Estado: em andamento** — a renovação do ERP interno já começou, sobre a fundação provada. *Uma frente do **Programa de Modernização IMAP**.*

O que é

O **SGF (Sistema de Gestão Financeira)** é a metade financeira do ERP que sustenta o dia a dia do Instituto: registra **vendas e contratos**, gera e controla **boletos e faturas**, concilia **depósitos e remessas bancárias** e calcula **comissões** de consultores e representantes. É onde o dinheiro do IMAP entra, é cobrado e é conciliado.

Nasceu na geração ColdFusion e agora recebe **a mesma base moderna já provada nas outras seis frentes** — a renovação já começou.

O tamanho da renovação (medido no código legado)

Números reais do código-fonte atual — a escala do trabalho:

Métrica	SGF hoje
Páginas <code>.cfm</code>	207
Linhas de código CFML	~49.200
Hub de roteamento único	<code>appSgf.cfm</code> — ~4.000 linhas num só arquivo
Front-end	template proprietário, sem build
Plataforma	ColdFusion + Windows Server — fora de suporte

O sistema por onde o dinheiro do Instituto entra, é cobrado e é conciliado **merece a mesma base moderna já provada nas outras frentes** — a modernização é ganho **financeiro, jurídico (LGPD) e de segurança**.

De → Para (proposto)

Dimensão	Antes (legado)	Depois (proposto)
Plataforma	ColdFusion (EOL)	Python 3 · FastAPI · open-source
Acesso a dados	camada legada	SQLAlchemy (ORM) — consultas parametrizadas por construção
Sistema operacional	Windows Server (EOL)	Linux + Docker
Front-end	template proprietário, sem build	Astro + Vite + TypeScript — build instantâneo, tipado
Banco	SQL Server (lado a lado na migração)	PostgreSQL (destino), lendo o mesmo banco durante o cutover
Licenciamento	CF + Windows (pago)	R\$ 0 (open-source)
Testes automatizados	—	pytest — regras financeiras e geração de boleto blindadas
Entrega	manual	CI + deploy containerizado
TLS/HTTPS	manual	Let's Encrypt automático

✓ Benefícios comuns (valem para todas as frentes do programa)

Detalhados no [Programa](#).

- 📄 **Licença de software R\$ 0** — Python, FastAPI, Docker e Linux open-source.
- 🛡️ **Fim de tecnologia sem suporte** — base mantida e com patches.
- 🐳 **Linux + Docker + Oracle Cloud** — o serviço `imap-sgf` entra no **host Docker consolidado**, junto das demais frentes.
- 🛡️ **Segurança em profundidade** — WAF, SIEM (Wazuh), TLS automático, hardening (SELinux), segredos protegidos.
- ♻️ **Migração incremental e reversível** — lê o mesmo banco, roda lado a lado, rollback rápido — **módulo a módulo**, como no SIEJ.
- 👥 **Mão de obra abundante** — Python é padrão de mercado; ColdFusion é cada vez mais raro no mercado.
- ⚙️ **Sem parar a operação.**

★ Benefícios específicos desta frente

- 🏠 **Integridade financeira por construção** — todas as escritas passam por **transações**, **validação de tipos (Pydantic)** e **consultas parametrizadas por padrão (SQLAlchemy)** — integridade garantida pela arquitetura, não por disciplina.
- 📄 **Boleto e remessa bancária sem componente proprietário** — geração de boleto/CNAB por bibliotecas abertas e testáveis, com **teste "golden"** que bloqueia o deploy se o arquivo de remessa mudar.
- 🔍 **Trilha de auditoria (LGPD)** — cada lançamento financeiro versionado e rastreável, alimentando o SIEM.
- ♿ **UX & acessibilidade** — telas financeiras responsivas, com foco visível e contraste.

🔍 Aprofundamento — os 5 eixos

- 🚀 **Modernização.** As ~49 mil linhas de CFML e o hub `appSgf.cfm` de ~4.000 linhas dão lugar a uma **API FastAPI** modular com front **Astro/Vite** tipado — a mesma arquitetura já em produção no DIOF e no SIEJ.
- 📦 **Custos.** Fim da **licença ColdFusion + Windows Server + CALs**; a stack nova é **R\$ 0** e o serviço **compartilha o host Docker** consolidado — menos VMs, menos fatura.
- 🛡️ **Endurecimento (hardening).** **Segredos protegidos, validação de configuração na inicialização, consultas parametrizadas por padrão** e contêiner Linux sob **SELinux**.
- 🛡️ **Segurança & salvaguarda.** O dinheiro público passa a rodar sobre **base mantida e monitorada: WAF + SIEM (Wazuh), TLS automático e testes que bloqueiam o deploy** (regras financeiras + golden do arquivo de remessa).
- ⌚ **Futuro já pavimentado.** O SGF **não começa do zero**: herda contêineres, CI, autenticação (JWT), WAF/SIEM/SELinux e o host consolidado. O método de cutover **módulo a módulo, lendo o mesmo banco, com rollback** já foi provado no SIEJ (~20 anos de legado).

📌 Estado & próximos passos

Esta frente — anunciada no [Pitch](#) (§6, *a renovação do ERP interno*) — **já está em andamento**: o serviço novo **já entrou no host Docker consolidado**, ao lado das demais frentes, herdando toda a fundação (CI, autenticação, WAF/SIEM/SELinux). O diagnóstico do legado está completo (números acima) e a renovação segue o **método provado** nas seis frentes anteriores: **módulo a módulo, lendo o mesmo banco, com rollback**.

🧰 Tecnologias

Python · FastAPI · SQLAlchemy · Astro · Vite · TypeScript · Docker · Linux/OCI · PostgreSQL · pytest · Let's Encrypt · CI — impacto de cada uma em [TECNOLOGIAS.md](#).

🗺️ Roadmap

1. **Fatiar o `appSgf.cfm`** em módulos de domínio (vendas, pagamentos, boletos, faturas, comissões) e cobrir cada um com testes de caracterização.

2. **Subir a API FastAPI lendo o mesmo banco**, módulo a módulo, lado a lado com o legado.
3. **Front Astro/Vite** substituindo as telas SmartAdmin, tela a tela.
4. **Cutover gradual + rollback**; ao final, desligar mais uma parcela do Windows legado.

Uma frente do Programa de Modernização IMAP. Sistemas críticos migrados de ColdFusion 11 + Windows Server 2012 R2 para uma base moderna, open-source e containerizada em nuvem — em produção, validada e reversível, com licença de software R\$ 0.